

Riassunto delle lezioni

L7 – L8 – L8b – L8c

Progettazione di applicazioni web full stack

Corso del Prof. Devis Bianchini – Università degli Studi di Brescia

Indice

1	L7 – Progettazione model-driven dei contenuti informativi (WebML)	2
1.1	Obiettivi della lezione	2
1.2	La metodologia WebML	3
1.3	Specifica dei requisiti	3
1.3.1	Raccolta dei requisiti	3
1.3.2	Analisi dei requisiti	3
1.4	Gerarchia dei gruppi utente	3
1.5	Requisiti sui dati e dizionario	4
2	L8 – Sviluppo frontend con React	4
2.1	Introduzione e contesto	4
2.1.1	Cos'è React.js	4
2.1.2	Approccio SPA	4
2.2	Concetto chiave: i componenti	4
2.3	File principali di un'app React in Nx	4
2.3.1	main.tsx	4
2.3.2	index.html	5
2.3.3	app.tsx	5
2.4	Componenti funzionali e prop	5
2.5	La sintassi JSX	6
2.6	Rendering condizionale	6
2.7	Liste	6
2.8	Eventi	6
2.9	Stato (state)	7
2.10	Condividere i dati tra componenti	7
2.11	Componenti e stile	7
2.11.1	1. Stile inline	7
2.11.2	2. File CSS classico	7
2.11.3	3. CSS Modules	7
2.11.4	4. Fogli di stile globali	7
2.11.5	5. dangerouslySetInnerHTML	8
2.12	Dietro le quinte: il Virtual DOM	8
2.12.1	Esempio concreto	8
2.13	Vantaggi di React	8
2.14	Riferimenti utili	8
3	L8b – Laboratorio React (catalogo libri + autenticazione)	8

3.1	Obiettivo della lezione	8
3.2	Procedura	8
3.3	Back-end: estrazione del catalogo	9
3.3.1	Service	9
3.3.2	Interfaccia condivisa	9
3.3.3	Controller	9
3.4	Cross-Origin Resource Sharing (CORS)	10
3.5	Front-end: routing principale	10
3.6	API client frontend	10
3.6.1	<code>auth.api.ts</code>	10
3.6.2	<code>books.api.ts</code>	11
3.7	Pagina di login	11
3.7.1	Stato e gestione del submit	11
3.7.2	Rendering con CSS Module	12
3.8	Pagina del catalogo	12
3.8.1	Pacchetto <code>clsx</code>	12
3.8.2	Pulsante di Logout nella header	13
3.8.3	Rendering della tabella	13
3.9	Componente di Logout	13
3.10	Proteggere una rotta	14
4	L8c – Approfondimenti React (CRUD completo + framework CSS)	14
4.1	Obiettivo della lezione	14
4.2	Estensioni della libreria online	15
4.2.1	Predisposizione della pagina “Nuovo libro”	15
4.2.2	Predisposizione di Modifica/Elimina nel catalogo	15
4.2.3	Predisposizione della pagina “Modifica libro”	15
4.2.4	Predisposizione delle navbar	15
4.3	Framework CSS – Tailwind	15
4.3.1	Installazione Tailwind in Nx + React + Vite	15
4.4	Framework CSS – Bootstrap	16
4.5	Confronto pratico: home page realizzata nei tre stili	16
4.5.1	Con CSS Modules	16
4.5.2	Con Tailwind	17
4.5.3	Con Bootstrap	17
4.6	Link utili	17
	Conclusioni	18

1. L7 – Progettazione model-driven dei contenuti informativi (WebML)

1.1 Obiettivi della lezione

La lezione introduce la metodologia **WebML** (*Web Modeling Language*) per la progettazione di applicazioni web *data-intensive*. L’output atteso è duplice:

- un modello dei dati progettato secondo i principi della metodologia;
- la codifica della logica di backend (con NestJS + TypeORM) guidata da tale modello.

1.2 La metodologia WebML

WebML è una metodologia di sviluppo per applicazioni web *data-intensive*, articolata in due assi principali:

- **Modellazione del contenuto informativo** (entità, proprietà, relazioni).
- **Modellazione dell'ipertesto** (struttura delle pagine, navigazione, interazione utente).

WebML guida sia la logica di backend sia la logica di frontend e rappresenta un passo avanti rispetto alla classica *Guida di Yale* (Lynch & Horton, 1999), che si limitava a indicare i passi di strategia, progettazione visiva, accessibilità, usabilità e manutenzione.

Il workflow WebML completo comprende:

1. Specifica dei requisiti (gruppi utente, casi d'uso, dizionario dei dati).
2. Progettazione dei dati.
3. Progettazione dell'ipertesto.
4. Implementazione.
5. Promozione, monitoraggio, valutazione, manutenzione.

1.3 Specifica dei requisiti

Input: requisiti di business. **Output:** specifiche semi-formali, sufficientemente precise ma comprensibili dal cliente. Si divide in due sotto-attività:

Raccolta dei requisiti

Attività poco strutturata, raccoglie:

- utenti e gruppi utente (interni/esterni);
- requisiti funzionali (casi d'uso);
- requisiti sui dati (dizionario);
- mockup dell'interfaccia web;
- requisiti non funzionali (usabilità, prestazioni, disponibilità, scalabilità, sicurezza, manutenibilità, multi-device).

Analisi dei requisiti

Revisione e formalizzazione dei requisiti raccolti tramite linguaggi appropriati:

- **UML** per gruppi di utenti e casi d'uso;
- soluzioni ad-hoc per dizionario dati e mockup dell'interfaccia.

1.4 Gerarchia dei gruppi utente

I gruppi utente si organizzano in gerarchia (es. *User* $\rightarrow$ *Registrato/NonRegistrato* $\rightarrow$ *Admin/Manager/...* oppure *Interno/Fornitore/Cliente*). Va sempre identificato **almeno un gruppo target** a cui dedicare l'applicazione, per focalizzare meglio le funzionalità. Si sfrutta il *meccanismo di ereditarietà* per descrivere casi d'uso comuni.

Per ogni gruppo si compila una scheda con: descrizione, dati di profilo, super-gruppo, sotto-gruppo, oggetti in lettura/scrittura, casi d'uso rilevanti.

1.5 Requisiti sui dati e dizionario

Tre domande tipiche guidano la raccolta:

1. Quali sono gli oggetti informativi da pubblicare/gestire?
2. Quali proprietà li caratterizzano?
3. In che modo i vari oggetti sono collegati?

Per ciascun elemento del dizionario si specifica: nome ed eventuali sinonimi, descrizione con istanze di esempio, proprietà (attributi), relazioni (incluse generalizzazione/specializzazione), componenti (concetto di *part-of*).

2. L8 – Sviluppo frontend con React

2.1 Introduzione e contesto

Cos'è React.js

React.js è una libreria JavaScript/TypeScript per lo sviluppo front-end di applicazioni web. Sviluppata nei laboratori Facebook nel 2011, usata per la prima volta su Instagram nel 2012, rilasciata pubblicamente nel 2013 con licenza MIT. Oggi è utilizzata da aziende come AirBnB, Netflix, Paypal, Uber.

Approccio SPA

React si ispira al paradigma delle **SPA (Single Page Applications)**:

- un unico *contenitore* che evolve dinamicamente con i dati dal back-end;
- nessun caricamento di nuove pagine dal server a ogni interazione;
- sviluppo per *componenti* che interagiscono con le API di React, le quali manipolano il DOM.

2.2 Concetto chiave: i componenti

In React le applicazioni sono fatte di **componenti**: oggetti dinamici che ricevono input dall'esterno (interazioni utente, time-out, comandi dal back-end) e producono l'elemento grafico da restituire.

Un componente è un “pezzo” di UI con:

- **logica**;
- **aspetto** (HTML o altri componenti annidati).

I componenti possono essere *piccoli* (un button) o *grandi* (intere pagine).

2.3 File principali di un'app React in Nx

main.tsx

Punto di ingresso che monta l'app nel DOM:

```
import { StrictMode } from 'react';
import { BrowserRouter } from 'react-router-dom';
import * as ReactDOM from 'react-dom/client';
import App from './app/app';

const root = ReactDOM.createRoot(
```

```

document.getElementById('root') as HTMLElement
);

root.render(
  <StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </StrictMode>
);

```

Elementi chiave:

- **StrictMode**: contenitore di sviluppo che segnala pratiche sconsigliate ed effetti collaterali; non aggiunge UI visibile.
- **BrowserRouter**: gestisce la navigazione SPA (cambio di URL come /, /about, /login senza ricaricare la pagina).
- **ReactDOM/client**: API moderna (React 18+) per manipolare il DOM.
- **ReactDOM.createRoot**: cerca in `index.html` l'elemento con `id="root"`, lo certifica come `HTMLElement` e crea una root React moderna.
- **App**: componente principale dell'applicazione (root component).

index.html

Entry point HTML che contiene il `<div id="root">` dove React monta i componenti, e lo `<script type="module" src="/src/main.tsx">`.

app.tsx

Componente radice. Importa altri componenti (es. `NxWelcome`) e li compone con il routing:

```

import NxWelcome from './nx-welcome';
import { Route, Routes, Link } from 'react-router-dom';

export function App() {
  return (
    <div>
      <NxWelcome title="@org/ui" />
      <Routes>
        <Route path="/" element={<div>Root</div>} />
      </Routes>
    </div>
  );
}

```

Routes è la parte dinamica: legge l'URL corrente e sostituisce la propria zona con l'HTML/-componente indicato nella `element`. Funziona perché **App** è dentro `<BrowserRouter>`.

2.4 Componenti funzionali e prop

Un componente funzionale è una funzione che ritorna JSX, esportata per essere usata altrove:

```

export function NxWelcome({ title }: { title: string }) {
  return <div>Welcome {title}</div>;
}
export default NxWelcome;

```

- Le **prop** sono i dati in input al componente; si destrutturano dall'oggetto argomento.
- Si tipizzano con TypeScript (`title: string`).
- `export default` è opzionale, ma comodo quando il file contiene un solo componente: permette `import NxWelcome from './nx-welcome'` senza parentesi graffe.

All'interno del JSX si visualizzano dati con la notazione a graffe `{...}`: prop, valori di stato, variabili locali.

2.5 La sintassi JSX

JSX (*JavaScript XML*) è HTML scritto all'interno di JavaScript. Permette di posizionare tag direttamente nel DOM senza chiamare `createElement()` o `appendChild()`. Regole:

- Componenti → notazione *camel-case* (`<MyComponent>`).
- HTML standard → minuscolo (`<div>`).
- Ogni tag di apertura deve avere uno di chiusura.

2.6 Rendering condizionale

La UI cambia in base a una condizione/stato:

```
let content;
if (isLoggedIn) {
  content = <AdminPanel />;
} else {
  content = <LoginForm />;
}
return <div>{content}</div>;
```

2.7 Liste

Si usa `map()` per generare elementi; ogni elemento richiede una **key** univoca:

```
const listItems = products.map(product =>
  <li
    key={product.id}
    style={{ color: product.isFruit ? 'magenta' : 'darkgreen' }}
  >
    {product.title}
  </li>
);
return <ul>{listItems}</ul>;
```

2.8 Eventi

React gestisce eventi come click, double-click, input. La sintassi è simile a JavaScript ma in JSX (camel-case):

```
function MyButton() {
  function handleClick() {
    alert('You clicked me!');
  }
  return <button onClick={handleClick}>Click me</button>;
}
```

2.9 Stato (state)

Lo stato è interno al componente e serve a:

- aggiornare la UI;
- reagire alle interazioni.

Quando lo stato cambia, React aggiorna la UI di conseguenza. Si usa l'hook `useState`:

```
import { useState } from 'react';

function MyButton() {
  const [count, setCount] = useState(0);
  function handleClick() {
    setCount(count + 1);
  }
  return (
    <button onClick={handleClick}>
      Clicked {count} times
    </button>
  );
}
```

2.10 Condividere i dati tra componenti

I dati possono essere:

- passati dall'alto come prop;
- condivisi sollevando lo stato al genitore comune (*lifting state up*).

2.11 Componenti e stile

Cinque approcci per dare uno stile ai componenti React:

1. *Stile inline*

Veloce ma poco leggibile. L'attributo `style={{ ... }}` accetta un oggetto JavaScript con proprietà in camel-case (`font-weight` → `fontWeight`), valori come stringhe.

2. *File CSS classico*

File `.css` separato importato con `import './style.css'`; nei componenti si usa `className` al posto di `class`. Semplice e standard, ma soggetto a conflitti di nome globali.

3. *CSS Modules*

File `.module.css` associato al componente. Approccio molto usato in Nx, niente conflitti di nome:

```
import styles from './nx-welcome.module.css';

<div className={styles.welcome}>...</div>
```

4. *Fogli di stile globali*

Linkati direttamente in `index.html` con `<link rel="stylesheet">`, validi per tutti i componenti, classi referenziate via `className`.

5. *dangerouslySetInnerHTML*

Approccio “non standard” e pericoloso. Forza React ad accettare HTML non filtrato (es. tag `<style>` inline). Esposto a vulnerabilità **XSS** (cross-site scripting); da evitare salvo necessità.

2.12 Dietro le quinte: il Virtual DOM

Quando si verifica un evento, React *non* manipola direttamente il DOM del browser come fanno le librerie tradizionali, ma manipola un **Virtual DOM**: una copia esatta del DOM mantenuta in memoria centrale. Più leggera da modificare, viene poi confrontata (operazione di *diffing*) con la copia precedente. React invia al DOM del browser *solo le modifiche strettamente necessarie*.

Esempio concreto

Lista di 10 check-box, utente clicca il primo:

- framework tradizionale: ri-renderizza tutti i 10 elementi.
- React: modifica solo l'elemento cambiato nel Virtual DOM, fa diffing, e invia al DOM del browser solo l'update del primo check-box (*rendering selettivo*).

2.13 Vantaggi di React

Per lo sviluppatore. L'approccio a componenti consente riuso e composizione di UI complesse da semplici *mattoncini*; lo sviluppatore si concentra sulla logica e sulla disposizione, mentre Virtual DOM e comunicazione col browser sono interamente a carico di React.

Per l'utente. Il rendering selettivo alleggerisce il processo di aggiornamento dell'interfaccia, con prestazioni percepibili migliori.

2.14 Riferimenti utili

- Documentazione ufficiale: <https://react.dev/learn>
- Convertitore HTML→JSX: <https://transform.tools/html-to-jsx>

3. L8b – Laboratorio React (catalogo libri + autenticazione)

3.1 Obiettivo della lezione

Creazione di una pagina front-end React per visualizzare un catalogo di libri, mettendo in comunicazione frontend e backend; la pagina è protetta da meccanismi di autenticazione e autorizzazione. Output: esempi di pagine React con relative rotte e un meccanismo di interazione completo tra le due tier.

3.2 Procedura

1. Predisporre una funzionalità di backend per estrarre il catalogo.
2. Abilitare la comunicazione tra frontend e backend (CORS).
3. Predisporre rotte e pagine nell'applicazione frontend.

3.3 Back-end: estrazione del catalogo

Service

Il file `books.service.ts` interroga il repository TypeORM e mappa l'entità in un DTO “flat” di interfaccia adatto al frontend:

```
async findAllBooks(): Promise<BookListItem[]> {
  const books = await this.bookRepository.find({
    relations: { authors: true, category: true },
    order: { title: 'ASC' }
  });

  return books.map((book) => ({
    id: book.id,
    title: book.title,
    publishedYear: book.publishedYear,
    category: { id: book.category.id, name: book.category.name },
    authors: book.authors.map((author) => ({
      id: author.id,
      firstName: author.firstName,
      lastName: author.lastName
    }))
  }));
}
```

Interfaccia condivisa

```
export interface BookListItem {
  id: number;
  title: string;
  publishedYear: number;
  category: { id: number; name: string; };
  authors: {
    id: number;
    firstName: string;
    lastName: string;
  }[];
}
```

Controller

Esponde la rotta protetta con `JwtAuthGuard` + `RolesGuard` e ritorna il DTO:

```
@ApiTags('Library APIs')
@Controller('books')
export class OrgBooksController {
  constructor(private orgBooksService: OrgBooksService) {}

  @Get()
  @UseGuards(JwtAuthGuard, RolesGuard)
  @Roles(UserRole.ADMIN)
  @ApiBearerAuth()
  findAll(): Promise<BookListItem[]> {
    return this.orgBooksService.findAllBooks();
  }
}
```

3.4 Cross-Origin Resource Sharing (CORS)

Per permettere a React (su localhost:4200) di chiamare NestJS (su localhost:3333) servono header CORS espliciti. Nel main.ts del backend:

```
async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableCors({
    origin: 'http://localhost:4200',
    credentials: true,
    allowedHeaders: ['Content-Type', 'Authorization'],
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS']
  });

  const globalPrefix = 'api';
  app.setGlobalPrefix(globalPrefix);
}
```

La configurazione permette:

- credenziali (cookie, certificati);
- header Content-Type per la login e Authorization per il JWT;
- tutti i verbi HTTP rilevanti per CRUD.

3.5 Front-end: routing principale

Il componente app.tsx definisce le rotte dell'applicazione:

```
import { BooksPage } from '../features/books/books.page';
import { Routes, Route, Navigate } from 'react-router-dom';
import { LoginPage } from '../features/auth/login.page';

export function App() {
  return (
    <Routes>
      <Route path="/" element={<Navigate to="/books" replace />} />
      <Route path="/login" element={<LoginPage />} />
      <Route path="/books" element={<BooksPage />} />
    </Routes>
  );
}
export default App;
```

3.6 API client frontend

auth.api.ts

Login: invio email/password, ricezione JWT, salvataggio in localStorage:

```
const API_URL = 'http://localhost:3333/api';

export async function login(email: string, password: string) {
  const response = await fetch(`${API_URL}/auth/login`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
  });
}
```

```

    if (!response.ok) {
      throw new Error('Credenziali non valide');
    }

    const data = await response.json();
    localStorage.setItem('access_token', data.access_token);
    return data;
  }

```

books.api.ts

Chiamata GET autenticata: header Authorization: Bearer <token>:

```

import { BookListItem } from './types';
const API_URL = 'http://localhost:3333/api';

export async function fetchBooks(): Promise<BookListItem[]> {
  const token = localStorage.getItem('access_token');

  const response = await fetch(`${API_URL}/books`, {
    headers: { Authorization: `Bearer ${token}` },
  });

  if (!response.ok) {
    throw new Error('Errore durante il caricamento dei libri');
  }

  return response.json();
}

```

3.7 Pagina di login

Stato e gestione del submit

Usa `useState` per email, password, errore e flag di loading; `useNavigate` per il redirect:

```

export function LoginPage() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const navigate = useNavigate();

  async function handleSubmit(e: React.FormEvent) {
    e.preventDefault();
    setError(null);
    setLoading(true);

    try {
      await login(email, password);
      navigate('/books');
    } catch (err: any) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
}

```

Rendering con CSS Module

Il foglio di stile (`books.module.css`) è condiviso tra le pagine. JSX semplificato:

```
import book_styles from '../css/books.module.css';

return (
  <main className={book_styles.page}>
    <div className={book_styles.card}>
      <h1 className={book_styles.title}>Library Login</h1>
      <form onSubmit={handleSubmit} className={book_styles.form}>
        <div className={book_styles.field}>
          <label>Email</label>
          <input
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
          />
        </div>
        <div className={book_styles.field}>
          <label>Password</label>
          <input
            type="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
          />
        </div>
        <button type="submit" disabled={loading}>
          {loading ? 'Accesso in corso...' : 'Login'}
        </button>
      </form>
      {error && <p className={book_styles.error}>{error}</p>}
    </div>
  </main>
);
```

3.8 Pagina del catalogo

Carica i libri al mount con `useEffect` e mostra stati di *loading*, *error* e *empty*:

```
export function BooksPage() {
  const [books, setBooks] = useState<BookListItem[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const navigate = useNavigate();

  useEffect(() => {
    fetchBooks()
      .then(setBooks)
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) { return <p>Caricamento libri...</p>; }
}
```

Pacchetto *clsx*

Permette di applicare più classi CSS Module insieme:

```
import clsx from 'clsx';
<section className={clsx(book_styles.card, book_styles.cardLarge)}>
```

Pulsante di Logout nella header

```
<button
  className={book_styles.logoutButton}
  onClick={() => navigate('/logout')}
>
  Logout
</button>
```

Rendering della tabella

Tabella con map sugli elementi e gestione di campi opzionali:

```
{books.length === 0 ? (
  <p>Nessun libro disponibile.</p>
) : (
  <table className={book_styles.table}>
    <thead>
      <tr>
        <th>Titolo</th><th>Anno</th>
        <th>Categoria</th><th>Autori</th>
      </tr>
    </thead>
    <tbody>
      {books.map((book) => (
        <tr key={book.id}>
          <td>{book.title}</td>
          <td>{book.publishedYear}</td>
          <td>
            <span>{book.category?.name ?? 'N/D'}</span>
          </td>
          <td>
            {book.authors?.length
              ? book.authors
                .map(a => `${a.firstName} ${a.lastName}`)
                .join(', ')
              : 'N/D'}
          </td>
        </tr>
      ))}
    </tbody>
  </table>
)}
```

3.9 Componente di Logout

Gestito interamente lato React, senza chiamate al back-end (il token JWT viene semplicemente rimosso dal localStorage):

```
import { useEffect } from 'react';
import { useNavigate } from 'react-router-dom';

export function LogoutPage() {
  const navigate = useNavigate();

  useEffect(() => {
```

```

    localStorage.removeItem('access_token');
    navigate('/login', { replace: true });
  }, [navigate]);

  return <p>Logout in corso...</p>;
}

```

3.10 Proteggere una rotta

Un componente aggiuntivo `ProtectedRoute` fa da middleware: se manca il token JWT, reindirige a `/login`; altrimenti renderizza i children:

```

import { ReactNode } from 'react';
import { Navigate } from 'react-router-dom';

export function ProtectedRoute({ children }: { children: ReactNode }) {
  const token = localStorage.getItem('access_token');

  if (!token) {
    return <Navigate to="/login" replace />;
  }

  return children;
}

```

Si applica nell'`App` avvolgendo le rotte protette:

```

<Routes>
  <Route path="/" element={<Navigate to="/books" replace />} />
  <Route path="/login" element={<LoginPage />} />
  <Route
    path="/books"
    element={
      <ProtectedRoute>
        <BooksPage />
      </ProtectedRoute>
    }
  />
  <Route path="/logout" element={<LogoutPage />} />
</Routes>

```

4. L8c – Approfondimenti React (CRUD completo + framework CSS)

4.1 Obiettivo della lezione

Estensione dell'esempio della Libreria online con aspetti tipici di un'applicazione web reale:

- pagina di creazione di un nuovo libro;
- pulsanti di modifica e cancellazione per ogni libro nel catalogo;
- pagina di modifica di un libro esistente;
- navbar condivisa.

In parallelo si introducono i framework CSS **Tailwind** e **Bootstrap**.

4.2 Estensioni della libreria online

Predisposizione della pagina “Nuovo libro”

Form con campi: Titolo, Anno pubblicazione, ID categoria (select), Autori (campo di ricerca con autocomplete + chip per gli autori selezionati). Bottone “Crea libro”.

Predisposizione di Modifica/Elimina nel catalogo

Per ogni riga della tabella, due bottoni: *Modifica* (naviga a `/books/:id/edit`) ed *Elimina* (`window.confirm` + `DELETE` + update ottimistico della lista).

Predisposizione della pagina “Modifica libro”

Stesso form della creazione, precompilato con `fetchBookById(id)` dentro un `useEffect`; submit con `PATCH`. Mostra validazioni inline tipo “Seleziona almeno un autore”.

Predisposizione delle navbar

Componente con i link alle pagine principali (Catalogo, Nuovo libro, Autori, Categorie) e dropdown utente con logout.

4.3 Framework CSS – Tailwind

Tailwind è un framework CSS *utility-first*: le utility descrivono direttamente lo stile, senza dover scrivere classi semantiche personalizzate.

```
/* CSS classico */
.card {
  background: white;
  padding: 2rem;
  border-radius: 12px;
}

/* Tailwind equivalente */
<div className="bg-white p-8 rounded-xl"></div>
```

Vantaggi:

- maggiore velocità di sviluppo;
- niente naming CSS, niente conflitti globali;
- design consistente;
- ideale per componenti React.

Installazione Tailwind in Nx + React + Vite

Tre passaggi:

1. Installazione pacchetti.

```
npm install -D tailwindcss@3 postcss autoprefixer
```

2. apps/ui/postcss.config.cjs.

```
module.exports = {
  plugins: {
```

```
tailwindcss: {},
autoprefixer: {},
},
};
```

3. apps/ui/tailwind.config.cjs.

```
module.exports = {
  content: ['./src/**/*.html', './src/**/*.js', './src/**/*.ts', './src/**/*.jsx', './src/**/*.tsx'],
  theme: { extend: {} },
  plugins: [],
};
```

4. Direttive Tailwind in styles.css.

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Il file `styles.css` è già importato in `main.tsx`.

4.4 Framework CSS – Bootstrap

Bootstrap è un framework CSS che fornisce *componenti già pronti* (button, card, navbar, modal, ecc.) oltre a utility classes. Più maturo e immediato di Tailwind ma meno flessibile/moderno.

Installazione minima:

```
npm install bootstrap
```

E import nel `main.tsx`:

```
import 'bootstrap/dist/css/bootstrap.min.css';
```

4.5 Confronto pratico: home page realizzata nei tre stili

Con CSS Modules

La pagina importa un `styles` dall'omonimo file `.module.css` e applica le classi locali:

```
import styles from '../css/home.module.css';

export function HomeModulePage() {
  return (
    <main className={styles.page}>
      <section className={styles.card}>
        <h1 className={styles.title}>Library App</h1>
        <p className={styles.description}>
          Benvenuto nel catalogo online della biblioteca.
        </p>
        <a className={styles.button} href="/books">
          Vai al catalogo
        </a>
      </section>
    </main>
  );
}
```


Con Tailwind

Niente file CSS dedicato: tutte le utility nelle `className`.

```
import { Link } from 'react-router-dom';

export function HomeTailwindPage() {
  return (
    <main className="flex min-h-screen items-center justify-center bg-slate-100 px-4">
      <section className="w-full max-w-md rounded-2xl bg-white p-10 text-center shadow-xl">
        <h1 className="text-3xl font-bold text-slate-900">Library App</h1>
        <p className="mt-4 text-slate-500">
          Benvenuto nel catalogo online della biblioteca.
        </p>
        <Link
          to="/books"
          className="mt-8 inline-block rounded-lg bg-blue-600 px-5 py-3 font-semibold text-white hover:bg-blue-700"
        >
          Vai al catalogo
        </Link>
      </section>
    </main>
  );
}
```

Con Bootstrap

Si usano le classi standard del framework (`card`, `btn btn-primary`, `d-flex`, ecc.):

```
import { Link } from 'react-router-dom';

export function HomeBootstrapPage() {
  return (
    <main className="min-vh-100 bg-light d-flex align-items-center justify-content-center px-3">
      <section
        className="card shadow p-5 text-center"
        style={{ maxWidth: 420 }}
      >
        <h1 className="fw-bold">Library App</h1>
        <p className="text-muted mt-3">
          Benvenuto nel catalogo online della biblioteca.
        </p>
        <Link to="/books" className="btn btn-primary mt-4">
          Vai al catalogo
        </Link>
      </section>
    </main>
  );
}
```

4.6 Link utili

- Documentazione Tailwind: <https://tailwindcss.com/>
- Icone per React: <https://react-icons.github.io/react-icons/>

Conclusioni

Le quattro lezioni costituiscono il percorso completo dal modello dei dati al frontend funzionante:

- **L7** fornisce il metodo (**WebML**) per derivare modello dati, gruppi utente e casi d'uso dai requisiti di business; questo è il punto di partenza che guida sia il backend sia il frontend.
- **L8** introduce **React**: SPA, componenti funzionali, JSX, state, eventi, rendering condizionale, liste, stili e Virtual DOM.
- **L8b** applica concretamente React costruendo un'app di catalogo libri completa di **autenticazione JWT**, **routing protetto**, comunicazione **CORS** con NestJS e logout lato client.
- **L8c** estende l'app con **CRUD completo** (creazione, modifica, eliminazione) e mostra a confronto tre approcci di stile: **CSS Modules**, **Tailwind** e **Bootstrap**.

Lo schema architetturale risultante è quello tipico delle SPA moderne: API REST autenticata via JWT (backend NestJS) + SPA React Router (frontend), con DTO condivisi tra le due tier per garantire coerenza dei tipi.